Computing Systems and Programming (3): iocontrol_serialout_fsm1

Dr. Clifford Choy

School of Design

15 Mar 2026

iocontrol_serialout_fsm1

iocontrol_serialout_fsm1

- Note changes to highlighted parts

Declare a variable `state`
Of type `int`
And initialized to value of `0`

A `switch` statement for defining what to do for each value of `state`
https://docs.arduino.cc/language-reference/en/structure/control-structure/switchCase/



Variable, Data Type and Operators (1)

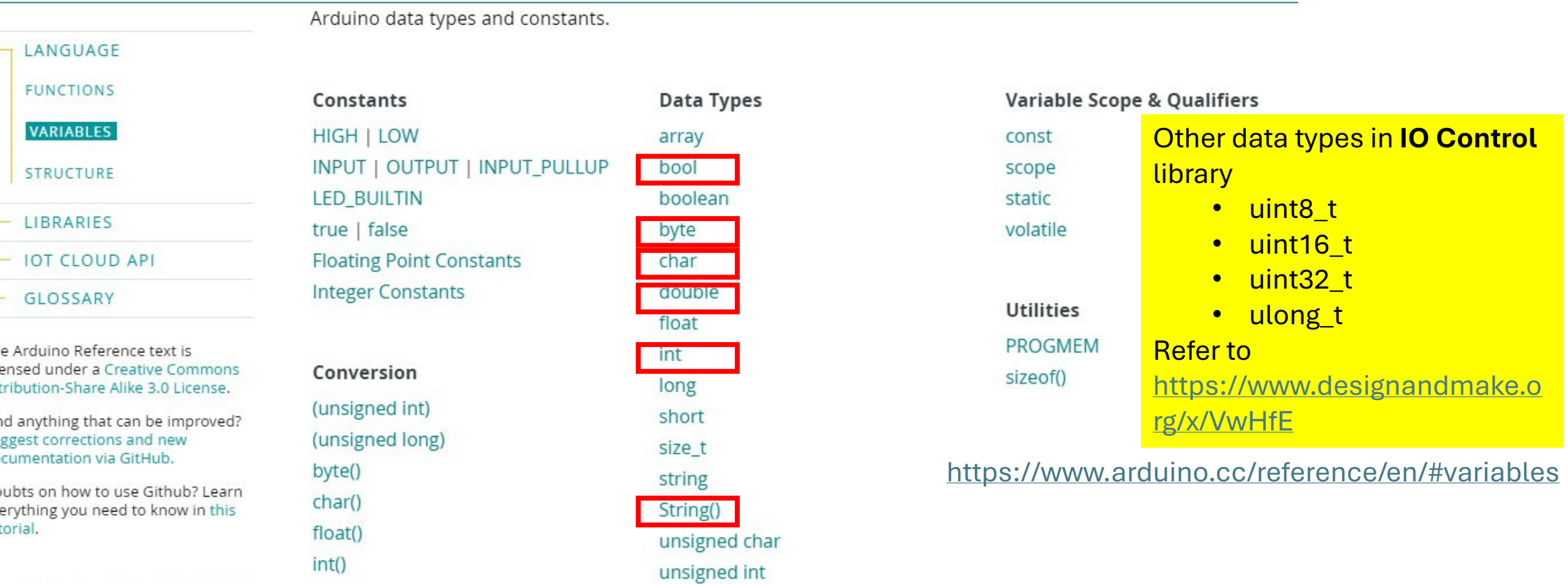
- A variable is for storing data in a program
- There are many variables in a program, and hence each variable needs a name for identification
- Rules on a variable name
  - Can consist of both uppercase (A-Z) and lowercase(a-z) letters
  - Can contain numbers 0 to 9, but cannot start with a number
  - May not have the same names as Arduino language keywords, e.g. you cannot have a variable named float
  - Must be unique, i.e. you cannot have two variables with the same name
  - Case sensitive, so `Count` and `count` are two different variables
  - Must not contain any special characters, except the underscore (_)

Variable, Data Type and Operators (2)

Valid Variable Names	Invalid Variable Names
counter	1counter
Counter	2Counter
_counter	3_counter
This_is_a_counter	$counter
_012345	%Count
c	0

Variable, Data Type and Operators (2)

- When declaring a variable, you must specify its **data type**



Other data types in **IO Control** library

- uint8_t
- uint16_t
- uint32_t
- ulong_t

Refer to https://www.designandmake.org/x/VwHfE

https://www.arduino.cc/reference/en/#variables

Variable, Data Type and Operators (3)

- `void`
  - nothing
- `bool`
  - a **Boolean** value, either `true` or `false`
- `byte`, `uint8_t`
  - an 8-bit **unsigned** value
  - from `0` to `255`
- `uint16_t`
  - an **unsigned** 16-bit value
  - From `0` to `65535`
- `int`
  - a **signed** 32-bit value
  - from `-2,147,483,648` to `2,147,483,647`
- `uint32_t`
  - an **unsigned** 32-bit value
  - from `0` to `4,294,967,295`
- `long`
  - A signed 64-bit value
  - from `-9223372036854775808` to `9223372036854775807`
- `ulong_t`
  - An unsigned 64-bit value
  - from `0` to `18446744073709551615`
- `char`
  - for storing one **ASCII-coded** character
  - A value of at least 8-bit (that is, with value from `0` to `255`)
- `double`
  - a number that has a decimal point
  - a **floating point** number or value
  - from -1.7 x 10^{308} (or `-1.7E+308`) to 1.7 x 10^{308} (or `1.7E+308`)
- `String`
  - a sequence of characters
  - E.g. `"Hello, World"`

Variable, Data Type and Operators (4)

Assignment operation
For assigning a value to the variable

```
bool var1 = true;    //  either "true" or "false"
byte var4 = 255;     //  from 0 to 255
char var5 = 'A';     //  a single character, hence use single quote
int var2 = 10;       //  an integer, and hence no decimal point
int var11 = -10;      //  can be negative integer as well
uint16_t var13 = 400000000; //  value too large
uint32_t var12 = -10; //  cannot store negative value
double var3 = 25.3;  //  value with decimal point
String var6 = "Hello, World\n"; //  a string, and use double quotes to enclose
```

Compile Example3_1.zip and observe results from Serial Monitor

ASCII Table

Everything in a computer is stored as a number

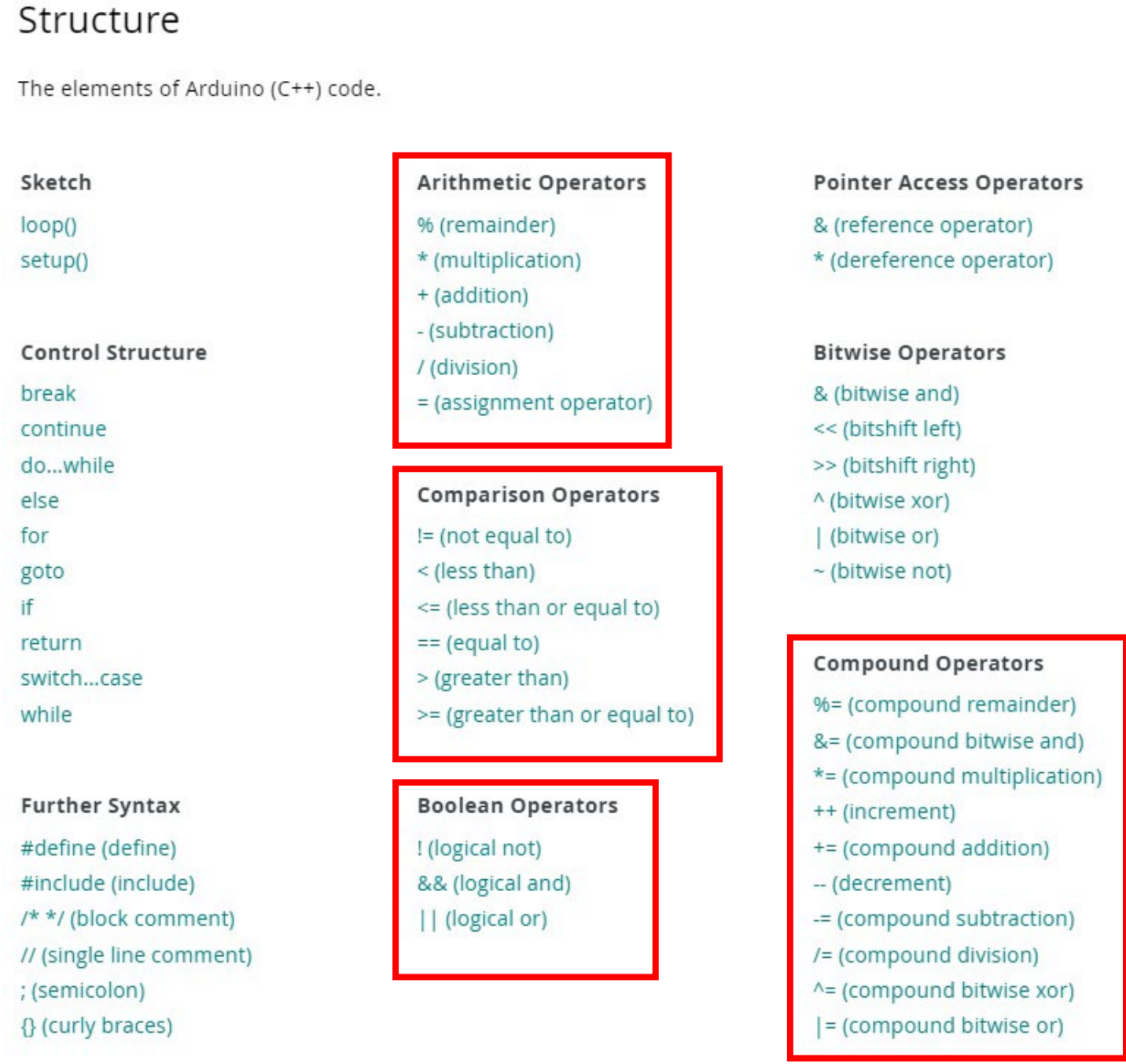
A character, e.g. `'a'` is stored as a number `97`

0	00	NUL	26	1A	SUB	52	34	4	78	4E	N	104	68	h
1	01	SOH	27	1B	ESC	53	35	5	79	4F	O	105	69	i
2	02	STX	28	1C	FS	54	36	6	80	50	P	106	6A	j
3	03	ETX	29	1D	GS	55	37	7	81	51	Q	107	6B	k
4	04	EOT	30	1E	RS	56	38	8	82	52	R	108	6C	l
5	05	ENQ	31	1F	US	57	39	9	83	53	S	109	6D	m
6	06	ACK	32	20	space	58	3A	:	84	54	T	110	6E	n
7	07	BEL	33	21	!	59	3B	;	85	55	U	111	6F	o
8	08	BS	34	22	"	60	3C	<	86	56	V	112	70	p
9	09	HT	35	23	#	61	3D	=	87	57	W	113	71	q
10	0A	LF	36	24	$	62	3E	>	88	58	X	114	72	r
11	0B	VT	37	25	%	63	3F	?	89	59	Y	115	73	s
12	0C	FF	38	26	&	64	40	@	90	5A	Z	116	74	t
13	0D	CR	39	27	'	65	41	A	91	5B	[	117	75	u
14	0E	SO	40	28	(	66	42	B	92	5C	\	118	76	v
15	0F	SI	41	29	)	67	43	C	93	5D	]	119	77	w
16	10	DLE	42	2A	*	68	44	D	94	5E	^	120	78	x
17	11	DC1	43	2B	+	69	45	E	95	5F	_	121	79	y
18	12	DC2	44	2C	,	70	46	F	96	60	`	122	7A	z
19	13	DC3	45	2D	-	71	47	G	97	61	a	123	7B	{
20	14	DC4	46	2E	.	72	48	H	98	62	b	124	7C	|
21	15	NAK	47	2F	/	73	49	I	99	63	c	125	7D	}
22	16	SYN	48	30	0	74	4A	J	100	64	d	126	7E	~
23	17	ETB	49	31	1	75	4B	K	101	65	e	127	7F	DEL
24	18	CAN	50	32	2	76	4C	L	102	66	f			
25	19	EM	51	33	3	77	4D	M	103	67	g			

https://www.rapidtables.com/code/text/ascii-table.html

Selected Operators

- Arithmetic
- Comparison
- Boolean
- Compound
- Click on each item in the page to find more details



Structure

The elements of Arduino (C++) code.

Sketch

loop()
setup()

Control Structure

break
continue
do...while
else
for
goto
if
return
switch...case
while

Further Syntax

#define (define)
#include (include)
/* */ (block comment)
// (single line comment)
; (semicolon)
{} (curly braces)

Arithmetic Operators

% (remainder)
* (multiplication)
+ (addition)
- (subtraction)
/ (division)
= (assignment operator)

Comparison Operators

!= (not equal to)
< (less than)
<= (less than or equal to)
== (equal to)
> (greater than)
>= (greater than or equal to)

Boolean Operators

! (logical not)
&& (logical and)
|| (logical or)

Pointer Access Operators

& (reference operator)
* (dereference operator)

Bitwise Operators

& (bitwise and)
<< (bitshift left)
>> (bitshift right)
^ (bitwise xor)
| (bitwise or)
~ (bitwise not)

Compound Operators

%= (compound remainder)
&= (compound bitwise and)
*= (compound multiplication)
++ (increment)
+= (compound addition)
-- (decrement)
-= (compound subtraction)
/= (compound division)
^= (compound bitwise xor)
|= (compound bitwise or)

https://www.arduino.cc/reference/en/#structure

Arithmetic Operators – Assignment

- Evaluate **expression** on the right of =, and store the result of the **expression** to the variable on the left of =

Arithmetic Operators

% (remainder)
* (multiplication)
+ (addition)
- (subtraction)
/ (division)
= (assignment operator)

https://www.arduino.cc/reference/en/#structure

```
int a=10, b=20, c;
double d, e=10.0, f=20.0;

c = a + b;   //   =30
c = a * b;   //   =200
c = a % b;   //   =10
c = a / b;   //   =0
d = double(a) / b ; //   = 0.5 (double)
d = a / double(b) ; //   = 0.5 (double)
d = e % f;  // compile time error
```

As a and b are integers, the division operator is operating in **integer mode**. Hence, value in c will be 0.

If at least one value is converted to floating point type, the operator will now be operating in **floating point mode** and hence can result in value with decimal point

operator % cannot work on values of **floating point** type, and hence error in compile time

Example3_2.ino

```
1  void setup() {
2    // put your setup code here, to run once:
3
4    int a=10, b=20, c;
5    double d, e=10.0, f=20.0;
6
7    c = a + b;  //  =30
8    c = a * b;  //  =200
9    c = a % b;  //  =10
10   c = a / b;  //  =0
11   d = double(a) / b ; //  = 0.5 (double)
12   d = a / double(b) ; //  = 0.5 (double)
13   d = e % f;  // compile time error
14
15 }
16
17 void loop() {
18   // put your main code here, to run repeatedly:
19
```

Compile Example3_2.zip and will generate the following error messages

Note that the statement with error is highlighted (line 13)

Output

```
D:\N\teach\sd\msc\25-26\sd5978\6. programming\programming_3\Example3_2\Example3_2.ino: In function 'void setup()
D:\N\teach\sd\msc\25-26\sd5978\6. programming\programming_3\Example3_2\Example3_2.ino:13:9: error: invalid opera
   13 |   d = e % f;  // compile time error
      |       ~ ^ ~
      |       |   |
      |       |   double
      |       double
exit status 1

Compilation error: invalid operands of types 'double' and 'double' to binary 'operator%'
```

About String (1)

- A sequence of characters
- `char`
  - Only one character
  - A value of at least 8-bit (that is, from `0` to `255`), according to ASCII code
- In Arduino program, the sequence of characters are enclosed by double quote (")
- **Escape sequence** is used for putting some special characters in the string, as they cannot be typed directly without affecting the program

ESCAPE SEQUENCE	DESCRIPTION	REPRESENTATION
`\'`	single quote	byte `0x27` in ASCII encoding
`\"`	double quote	byte `0x22` in ASCII encoding
`\?`	question mark	byte `0x3f` in ASCII encoding
`\\`	backslash	byte `0x5c` in ASCII encoding
`\a`	audible bell	byte `0x07` in ASCII encoding
`\b`	backspace	byte `0x08` in ASCII encoding
`\f`	form feed – new page	byte `0x0c` in ASCII encoding
`\n`	line feed – new line	byte `0x0a` in ASCII encoding
`\r`	carriage return	byte `0x0d` in ASCII encoding
`\t`	horizontal tab	byte `0x09` in ASCII encoding
`\v`	vertical tab	byte `0x0b` in ASCII encoding
`\nnn`	arbitrary octal value	byte nnn
`\xnn`	arbitrary hexadecimal value	byte nn
`\unnnn` (since C++11)	universal character name (arbitrary Unicode value); may result in several characters	code point U+nnnn
`\Unnnnnnnn` (since C++11)	universal character name (arbitrary Unicode value); may result in several characters	code point U+nnnnnnnn

About String (2)

- Frequently used escape character
  - \'
  - \"
  - \\
  - \n
- Although each “escape character” consists of two characters, they are interpreted as just one character

```
String var7 = "Hello\n";  // 6 characters in this string
String var8 = "Hello\nThis\nis\na\nTest\n"; //  \n means new line
String var9 = "This is \"School of Design\""; //  \" means a double quote character
String var10 = "This is Clifford\'s cat\n";   //  \' means a signle quote character
```

Compile Example3_3.zip and observe results from Serial Monitor

iocontrol_serialout_fsm1

Can be a block (actually prefer to define a block for grouping these statements)

go to this point when `state` is equal to `0`

Must have a `break` statement

go to this point when `state` does not match with all those in `case`

iocontrol_serialout_fsm1.ino

```
14  DeclareHandler(hdlr,_ctrl,IOControl,{ //  declare a handler for sc
15    //  hdlr - name of the handler function
16    //  _ctrl - Control object
17    //  IOControl - class of object of this handler
18    //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is
19    //
20
21    // if (_ctrl.isSchedulerTimeEvent())
22    //     Serial.println("IOControl - SchedulerTimerEvent");
23    Serial.print("current time=");
24    Serial.println(_ctrl.getUpdateTime());
25    Serial.print(" state=");
26    Serial.println(state);
27
28    //  a switch statement for specifying actions for each value of
29    //  actions:
30    //    print to Serial
31    //    set new value of "state"
32    switch (state)
33    { case 0:
34        Serial.println("hello");
35        state=1;
36        break;
37      case 1:
38        Serial.println("world");
39        state=2;
40        break;
41      case 2:
42        Serial.println("!!!");
43        state=0;
44        break;
45      default:
46        Serial.println("not possible!!!");
47        break;
48    }
49  });
```

Output Serial Monitor

Message (Enter to send message to 'AirM2M_

```
chip started
current time=100 state=0
hello
current time=1100 state=1
world
current time=2100 state=2
!!!
current time=3100 state=0
hello
current time=4100 state=1
world
current time=5100 state=2
!!!
current time=6100 state=0
hello
current time=7100 state=1
world
current time=8100 state=2
!!!
current time=9100 state=0
hello
current time=10100 state=1
world
current time=11100 state=2
!!!
current time=12100 state=0
hello
current time=13100 state=1
world
current time=14100 state=2
!!!
current time=15100 state=0
```

iocontrol_serialout_fsm1

- handler function is invoked once every 1000ms
- Value of variable `state` is changed every 1000ms
  - 0 (initialized value)
  - 1
  - 2
  - 0
  - 1
  - …
- Statements in `default` should never be triggered in the given program

iocontrol_serialout_fsm2

- Use Finite State Machine
- Added `repeatCount` to allow for repeating states for a few times

```
12  int state=0;  //  define a variable "state" with initial value of 0
13                //  "state" is for indicating which message to print in the sequence of messag
14  int repeatCount=0;  //  variable "repeatCount" is for determine when to end the sequence of
15
16  DeclareHandler(hdlr,_ctrl,IOControl,{ //  declare a handler for scheduler
17    //  hdlr - name of the handler function
18    //  _ctrl - Control object
19    //  IOControl - class of object of this handler
20    //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is invoked
21    //
22
23    // if (_ctrl.isSchedulerTimeEvent())
24    //     Serial.println("IOControl - SchedulerTimerEvent");
25    Serial.print("current time=");
26    Serial.println(_ctrl.getUpdateTime());
27    Serial.print(" state=");
28    Serial.println(state);
29
30    switch (state)
31    { case 0:
32        Serial.println("hello");
33        state=1;
34        break;
35      case 1:
36        Serial.println("world");
37        state=2;
38        break;
39      case 2:
40        Serial.println("!!!");
41        repeatCount++;      //  same as "repeatCount = repeatCount+1"
42        if (repeatCount<=3)
43        { state=0;          //  repeat the sequence in printing "hello", "world", "!!!"
44          Serial.print("repeated ");
45          Serial.print(repeatCount);
46          Serial.println(" times");
47        } else
48        { state=-1; //  stop the sequence
49          Serial.println("no more printing!!!");
50          repeatCount = 0;
51        }
52        break;
53      default:
54        //  do nothing
55        break;
56    }
57  });
```

Task 2

- Modify the program “iocontrol_serialout_fsm1” so that it prints out the following:

```
At t = 2000, Hello
At t = 3000, from
At t = 3500, “School
At t = 4000, of
At t = 4500, Design”
At t = 5500, !!!
```

- Compile and upload your program to ESP32C3

- Under the heading “Task 2” in your submission page, upload the following:
  - Screen-capture of the output of your program from Serial Monitor
  - Compressed file of your sketch (in zip format)

Summary

- Finite state machine (FSM)
- Variable name
- Data types
- Assignment operator (=)
- `switch` … `case` … `default` … `break`